Desafio 2 (UdeABook)

Integrantes:

Pablo Colorado Correa

Andrea Julieth Arias Cantillo

Análisis del problema:

El Desafío 2, denominado UdeABook, consiste en simular una red social mediante una aplicación que funciona por consola. En esta plataforma, los usuarios pueden interactuar entre sí a través de diferentes funcionalidades, como agregar amigos, enviar mensajes, consultar fotografías y recibir sugerencias de amistad. Toda la información se encuentra almacenada en archivos de texto (.txt), y los cambios realizados durante la ejecución del programa se reflejan en dichos archivos al momento de salir de la aplicación.

Diseño de la solución:

Para abordar el problema, lo primero que hicimos fue leer detalladamente el documento del desafío, con el objetivo de comprender los requerimientos e identificar las clases necesarias para desarrollar el sistema. Inicialmente, contemplamos clases como Usuario, Album, Foto, Mensajes y SolicitudAmistad, entre otras. Sin embargo, a medida que avanzamos en el desarrollo, identificamos la necesidad de incorporar clases adicionales para organizar mejor las responsabilidades y facilitar el funcionamiento del programa.

Para gestionar los diferentes conjuntos de datos, implementamos una clase genérica denominada Lista, utilizando plantillas (templates) de C++. Esta decisión nos permitió reutilizar una misma estructura para almacenar objetos de diferentes tipos, evitando la necesidad de implementar una lista independiente para cada clase. De esta manera, conseguimos una solución más organizada y flexible.

Análisis de eficiencia:

Una de las principales dificultades del desafío fue la restricción del uso de la biblioteca STL, lo que nos obligó a construir nuestras propias estructuras de datos y a administrar manualmente aspectos relacionados con la memoria.

Durante el desarrollo, identificamos que las relaciones de amistad se consultaban en múltiples funcionalidades, como la generación de sugerencias, el envío de solicitudes de amistad y la selección de fotografías de perfil. Debido a la frecuencia de estas consultas, decidimos implementar una matriz de amistades para facilitar el acceso a esta información y reducir el costo de las consultas.

No obstante, reconocemos que esta decisión presenta limitaciones cuando aumenta considerablemente la cantidad de usuarios, debido al consumo de memoria que puede generar una matriz de este tipo. Por esta razón, consideramos que uno de los principales aspectos por mejorar en una futura actualización sería la implementación de una estructura más eficiente para representar las relaciones entre usuarios.

Problemas de desarrollo:

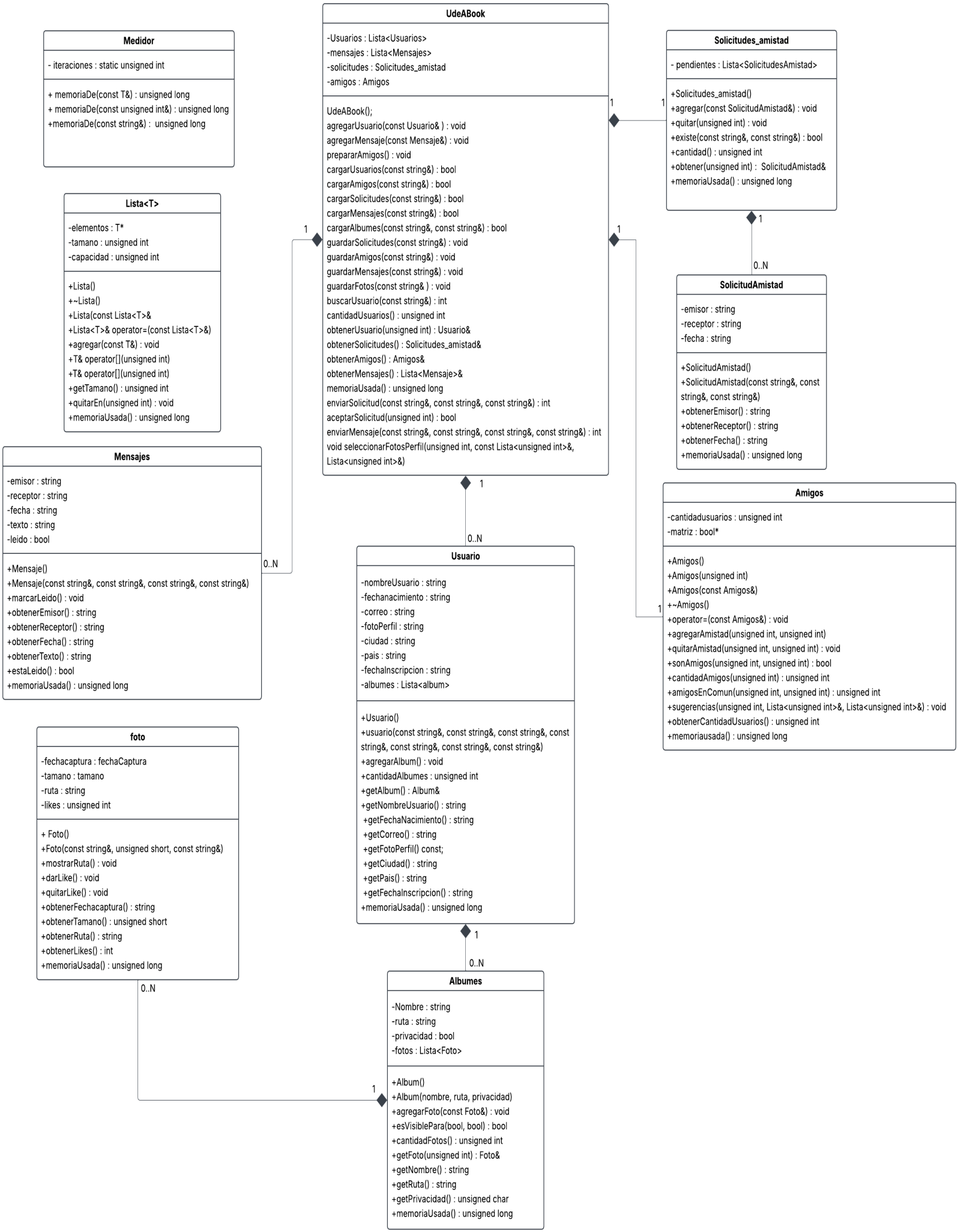
A lo largo del desarrollo nos enfrentamos a diferentes dificultades. Una de ellas fue la necesidad de crear más clases de las que habíamos previsto inicialmente, debido a la complejidad y a la cantidad de funcionalidades que debía cumplir el sistema.

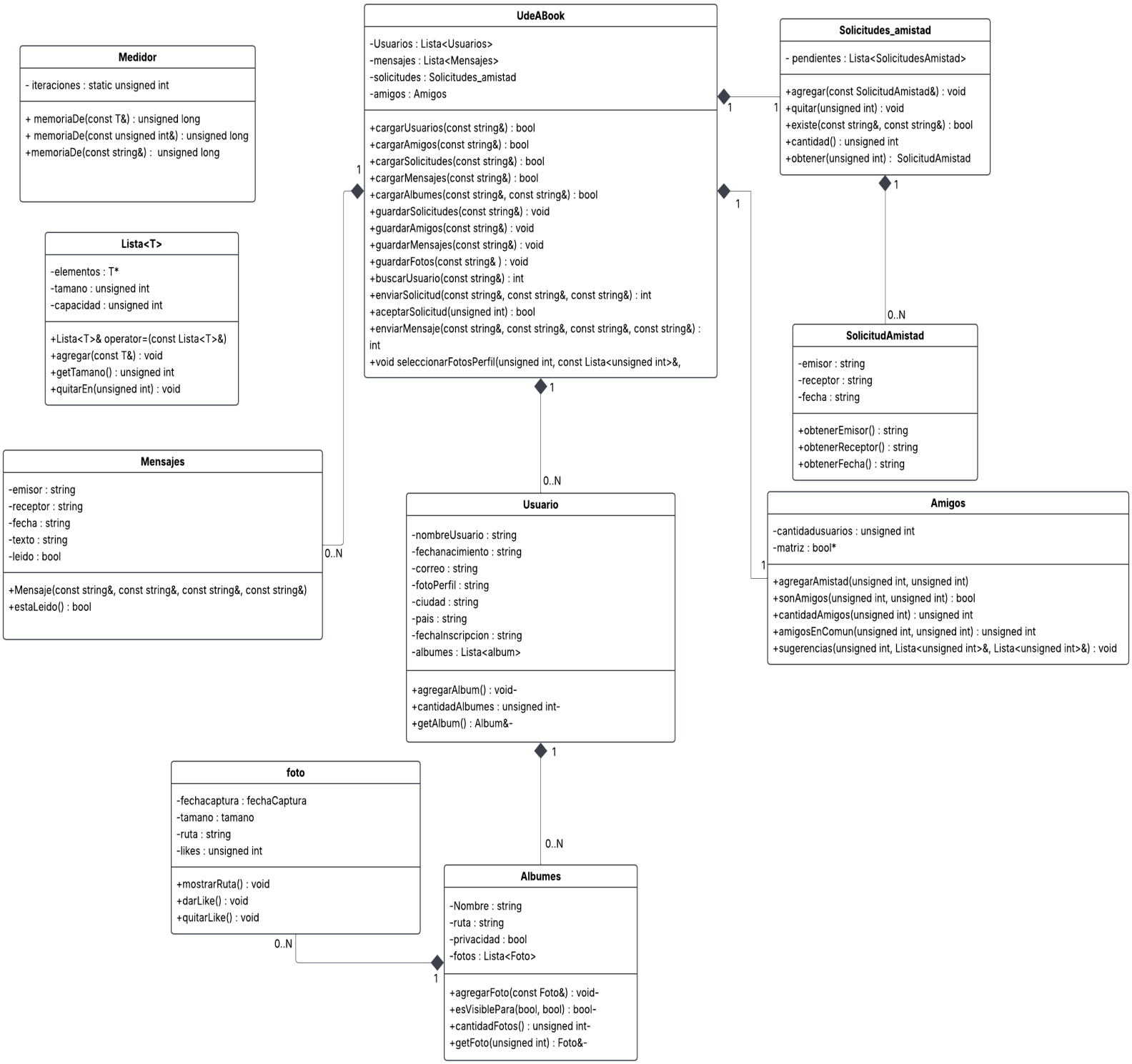
Por otra parte, la extensión del código hizo que la identificación y corrección de algunos errores resultara más complicada. También se presentaron errores menores relacionados con el manejo de ciertos datos, que fue necesario revisar y corregir durante las pruebas.

Estas dificultades nos permitieron reconocer la importancia de mantener una estructura organizada, distribuir adecuadamente las responsabilidades entre las clases y realizar pruebas constantes a medida que se incorporan nuevas funcionalidades.

Link Video: https://www.youtube.com/watch?v=1oTzn2eQHhY

Link Repositório: https://github.com/pcoloradocorrea-byte/Desafio-2-UdeABook..git


Medidor
- iteraciones : static unsigned int
+ memoriaDe(const T&) : unsigned long
+ memoriaDe(const unsigned int&) : unsigned long
+memoriaDe(const string&) : unsigned long
UdeABook
-Usuarios : Lista<Usuarios>
-mensajes : Lista<Mensajes>
-solicitudes : Solicitudes_amistad
-amigos : Amigos
UdeABook();
agregarUsuario(const Usuario&) : void
agregarMensaje(const Mensaje&) : void
prepararAmigos() : void
cargarUsuarios(const string&) : bool
cargarAmigos(const string&) : bool
cargarSolicitudes(const string&) : bool
cargarMensajes(const string&) : bool
cargarAlbumes(const string&, const string&) : bool
guardarSolicitudes(const string&) : void
guardarAmigos(const string&) : void
guardarMensajes(const string&) : void
guardarFotos(const string&) : void
buscarUsuario(const string&) : int
cantidadUsuarios() : unsigned int
obtenerUsuario(unsigned int) : Usuario&
obtenerSolicitudes() : Solicitudes_amistad&
obtenerAmigos() : Amigos&
obtenerMensajes() : Lista<Mensaje>&
memoriaUsada() : unsigned long
enviarSolicitud(const string&, const string&, const string&) : int
aceptarSolicitud(unsigned int) : bool
enviarMensaje(const string&, const string&, const string&, const string&) : int
void seleccionarFotosPerfil(unsigned int, const Lista<unsigned int>&, Lista<unsigned int>&)
Solicitudes_amistad
- pendientes : Lista<SolicitudesAmistad>
+Solicitudes_amistad()
+agregar(const SolicitudAmistad&) : void
+quitar(unsigned int) : void
+existe(const string&, const string&) : bool
+cantidad() : unsigned int
+obtener(unsigned int) : SolicitudAmistad&
+memoriaUsada() : unsigned long
SolicitudAmistad
-emisor : string
-receptor : string
-fecha : string
+SolicitudAmistad()
+SolicitudAmistad(const string&, const string&, const string&)
+obtenerEmisor() : string
+obtenerReceptor() : string
+obtenerFecha() : string
+memoriaUsada() : unsigned long
Lista<T>
-elementos : T*
-tamano : unsigned int
-capacidad : unsigned int
+Lista()
+~Lista()
+Lista(const Lista<T>&
+Lista<T>& operator=(const Lista<T>&)
+agregar(const T&) : void
+T& operator[](unsigned int)
+T& operator[](unsigned int)
+getTamano() : unsigned int
+quitarEn(unsigned int) : void
+memoriaUsada() : unsigned long
Mensajes
-emisor : string
-receptor : string
-fecha : string
-texto : string
-leido : bool
+Mensaje()
+Mensaje(const string&, const string&, const string&, const string&)
+marcarLeido() : void
+obtenerEmisor() : string
+obtenerReceptor() : string
+obtenerFecha() : string
+obtenerTexto() : string
+estaLeido() : bool
+memoriaUsada() : unsigned long
Usuario
-nombreUsuario : string
-fechanacimiento : string
-correo : string
-fotoPerfil : string
-ciudad : string
-pais : string
-fechaInscripcion : string
-albumes : Lista<album>
+Usuario()
+usuario(const string&, const string&, const string&, const string&, const string&, const string&, const string&)
+agregarAlbum() : void
+cantidadAlbumes : unsigned int
+getAlbum() : Album&
+getNombreUsuario() : string
+getFechaNacimiento() : string
+getCorreo() : string
+getFotoPerfil() const;
+getCiudad() : string
+getPais() : string
+getFechaInscripcion() : string
+memoriaUsada() : unsigned long
Amigos
-cantidadusuarios : unsigned int
-matriz : bool*
+Amigos()
+Amigos(unsigned int)
+Amigos(const Amigos&)
+~Amigos()
+operator=(const Amigos&) : void
+agregarAmistad(unsigned int, unsigned int)
+quitarAmistad(unsigned int, unsigned int) : void
+sonAmigos(unsigned int, unsigned int) : bool
+cantidadAmigos(unsigned int) : unsigned int
+amigosEnComun(unsigned int, unsigned int) : unsigned int
+sugerencias(unsigned int, Lista<unsigned int>&, Lista<unsigned int>&) : void
+obtenerCantidadUsuarios() : unsigned int
+memoriausada() : unsigned long
foto
-fechacaptura : fechaCaptura
-tamano : tamano
-ruta : string
-likes : unsigned int
+ Foto()
+Foto(const string&, unsigned short, const string&)
+mostrarRuta() : void
+darLike() : void
+quitarLike() : void
+obtenerFechacaptura() : string
+obtenerTamano() : unsigned short
+obtenerRuta() : string
+obtenerLikes() : int
+memoriaUsada() : unsigned long
Albumes
-Nombre : string
-ruta : string
-privacidad : bool
-fotos : Lista<Foto>
+Album()
+Album(nombre, ruta, privacidad)
+agregarFoto(const Foto&) : void
+esVisiblePara(bool, bool) : bool
+cantidadFotos() : unsigned int
+getFoto(unsigned int) : Foto&
+getNombre() : string
+getRuta() : string
+getPrivacidad() : unsigned char
+memoriaUsada() : unsigned long
1
0..N


Medidor
- iteraciones : static unsigned int
+ memoriaDe(const T&) : unsigned long
+ memoriaDe(const unsigned int&) : unsigned long
+memoriaDe(const string&) : unsigned long
Lista<T>
-elementos : T*
-tamano : unsigned int
-capacidad : unsigned int
+Lista<T>& operator=(const Lista<T>&)
+agregar(const T&) : void
+getTamano() : unsigned int
+quitarEn(unsigned int) : void
Mensajes
-emisor : string
-receptor : string
-fecha : string
-texto : string
-leido : bool
+Mensaje(const string&, const string&, const string&, const string&)
+estaLeido() : bool
UdeABook
-Usuarios : Lista<Usuarios>
-mensajes : Lista<Mensajes>
-solicitudes : Solicitudes_amistad
-amigos : Amigos
+cargarUsuarios(const string&) : bool
+cargarAmigos(const string&) : bool
+cargarSolicitudes(const string&) : bool
+cargarMensajes(const string&) : bool
+cargarAlbumes(const string&, const string&) : bool
+guardarSolicitudes(const string&) : void
+guardarAmigos(const string&) : void
+guardarMensajes(const string&) : void
+guardarFotos(const string&) : void
+buscarUsuario(const string&) : int
+enviarSolicitud(const string&, const string&, const string&) : int
+aceptarSolicitud(unsigned int) : bool
+enviarMensaje(const string&, const string&, const string&, const string&) : int
+void seleccionarFotosPerfil(unsigned int, const Lista<unsigned int>&,
Solicitudes_amistad
- pendientes : Lista<SolicitudesAmistad>
+agregar(const SolicitudAmistad&) : void
+quitar(unsigned int) : void
+existe(const string&, const string&) : bool
+cantidad() : unsigned int
+obtener(unsigned int) : SolicitudAmistad
SolicitudAmistad
-emisor : string
-receptor : string
-fecha : string
+obtenerEmisor() : string
+obtenerReceptor() : string
+obtenerFecha() : string
Usuario
-nombreUsuario : string
-fechanacimiento : string
-correo : string
-fotoPerfil : string
-ciudad : string
-pais : string
-fechaInscripcion : string
-albumes : Lista<album>
+agregarAlbum() : void-
+cantidadAlbumes : unsigned int-
+getAlbum() : Album&-
Amigos
-cantidadusuarios : unsigned int
-matriz : bool*
+agregarAmistad(unsigned int, unsigned int)
+sonAmigos(unsigned int, unsigned int) : bool
+cantidadAmigos(unsigned int) : unsigned int
+amigosEnComun(unsigned int, unsigned int) : unsigned int
+sugerencias(unsigned int, Lista<unsigned int>&, Lista<unsigned int>&) : void
foto
-fechacaptura : fechaCaptura
-tamano : tamano
-ruta : string
-likes : unsigned int
+mostrarRuta() : void
+darLike() : void
+quitarLike() : void
Albumes
-Nombre : string
-ruta : string
-privacidad : bool
-fotos : Lista<Foto>
+agregarFoto(const Foto&) : void-
+esVisiblePara(bool, bool) : bool-
+cantidadFotos() : unsigned int-
+getFoto(unsigned int) : Foto&-
1
0..N